Sift Technical Design Document

AI resume screening tool and client shortlist portal for Company A, Noida

GitHub: https://github.com/<your-username>/sift

Live app: <add the deployed URL here if you deploy it>

Version: 1.0, November 2026. Run instructions and demo logins are in `deployment.md`.

1. Summary

Company A's recruiters read about 1,800 hours of CVs a month, and slow shortlists lose placements. The brief puts the cost at ₹1.57 crore a year (₹75.6 lakh of reading time plus ₹81 lakh of lost fees). Sift lets a recruiter upload a pile of CVs against a job and see the best first, **with the exact line from each CV that earned the score**. Clients review shortlists on their phone. Every view of personal data is logged.

The three targets in the brief are: screening time per requisition down 60%, a shortlist within 48 hours for 80% of requisitions, and client acceptance up from 45% to 60%. The Insights page measures all three (section 11).

> **The design in one paragraph**
> One Node service and one SQLite file. A deterministic, explainable scoring engine ranks CVs and quotes its evidence. It never rejects anyone, hides protected attributes before scoring, and a recruiter must open a CV before it can go to a client. The React app has a recruiter workspace, a phone-first client portal, and a delivery-head view.

1.1 Coverage of the brief

Must-have in the brief	Status	How it is covered
JD intake into an editable scorecard	Built	Paste a JD. Skills, experience band, notice limit, location and budget are extracted into a scorecard the recruiter edits (Must or Nice, weight 1 to 3). Rules by default, optional LLM.
Bulk CV upload (PDF or DOCX, up to 500), parsing, duplicates	Built	Sent in batches of 20 for real progress. PDF, DOCX and TXT. Duplicates matched on email, phone or file hash. Scans are flagged, not OCR'd.
AI score per CV with quoted evidence	Built	Each criterion carries a verbatim CV line, highlighted in the CV view.
Client portal with masked contacts: shortlist, reject with reason, propose slot	Built	Mobile-first. Names shown as first name and initial. Reason required to reject. Slot must be in the future.
Pipeline per requisition, sourced to joined	Built	Kanban board with server-enforced transition rules.
Candidate consent record and deletion on request	Built	Consent stored per candidate. Erasure removes personal data and the file.

Must-have in the brief	Status	How it is covered
Log of who viewed what (DPDP)	Built	Audit log of profile views, CV opens, file opens, client views, decisions and erasures.
Weekly sample of AI ranks vs recruiter choices	Built	On the delivery-head page, computed on demand over the last 7 days.

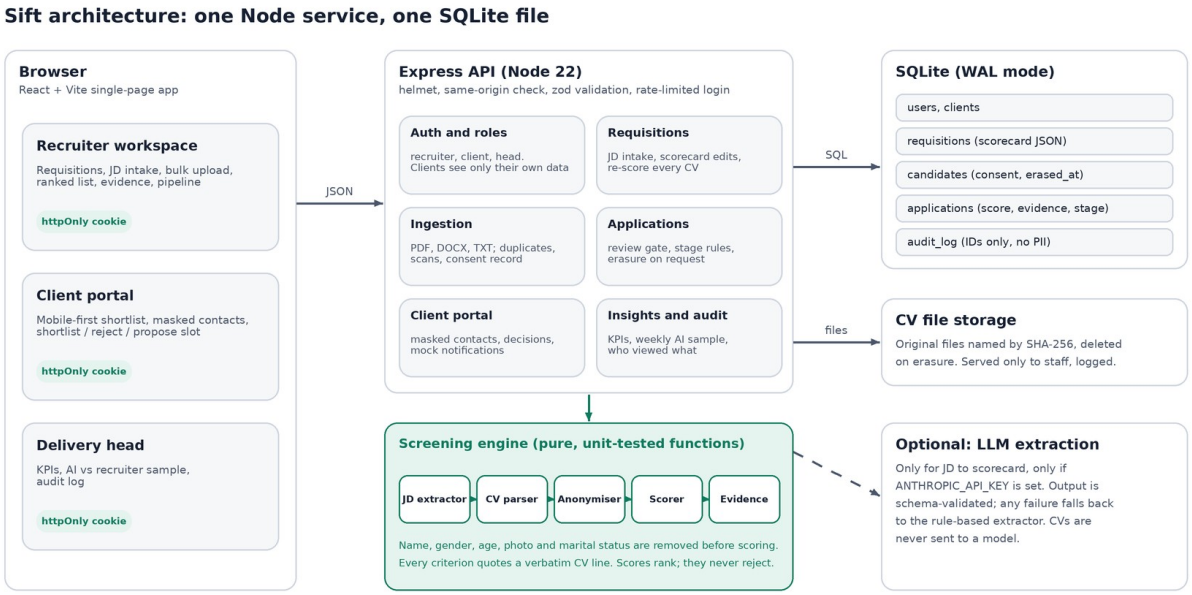
2. Assumptions

The brief says to use only the information in the problem statement and to label every assumption. These are all of them.

ID	Assumption	Effect if wrong
A1	Scale and baseline: 300 requisitions a month, about 120 CVs each, 3 minutes to read a CV (all from the brief). Used only to estimate reading time saved.	The time-saved figure changes. Nothing else does.
A2	Most CVs are English, text-based PDF or DOCX. Some are scans (stated in the brief).	Scans are flagged and unscored, not OCR’d (see section 10).
A3	Scoring weights (skills 50, bonus skills 10, experience 20, notice 10, location 5, budget 5) and bands (75 and 50) are my judgement. There is no labelled data to calibrate them.	Rankings shift. Weights live in one file and are shown to users.
A4	A notice period up to 30 days over the limit earns half credit, because buy-outs are common.	Strict clients may want a harder cut-off.
A5	Noida, Gurugram, Delhi, Ghaziabad and Faridabad are one commuting region for the location criterion.	Location credit changes for nearby cities.
A6	Candidates consented before upload. The recruiter records how consent was obtained. There is no candidate-facing consent link.	A real rollout needs candidate-side consent capture.
A7	Clients see first name and last initial plus masked email and phone. Full name and contacts stay with Company A. The brief only says “masked contacts”.	Easy to change in one function.
A8	Clients may read the CV text with email, phone and profile links removed, and see the score with its evidence.	Could be restricted to summary only.
A9	The delivery head is read-only. Only recruiters change data.	Needs a role change if heads must act.
A10	A candidate submitted to a client is recruiter-approved, so “submitted” means “reviewed”.	The review gate (open the CV first) enforces this.
A11	One scorecard per requisition. Editing it re-scores every CV and leaves stages unchanged.	Versioned scorecards would be needed for audit of old scores.
A12	Erasure on request removes personal data and the stored file. An anonymous row and an ID-only audit entry remain. This is my reading of the DPDP Act 2023, not legal advice.	Counsel should confirm retention rules.

ID	Assumption	Effect if wrong
A13	Money is in LPA (lakhs per annum). Notice is in days.	Parser and labels use these units.
A14	Messages are mocked, as the brief allows. Client decisions create a logged “mock notification” instead of an email.	Real email is a small addition.
A15	SQLite on one disk is enough for a pilot with about 70 recruiters.	PostgreSQL is the scale path (section 3).
A16	The weekly sample is computed on request for the last 7 days, not by a scheduled job. Demo data and all candidates are fictional.	A cron job would send it by email.

3. Architecture and why



The budget is ₹7 lakh and six weeks, and a reviewer must be able to run the product in minutes. That pushed every choice toward **fewer moving parts**.

Decision	Choice	Why, and the trade-off
Shape	One Node service that serves the API and the built React app	One thing to deploy and one place for logs. Trade-off: scales by instance, not by component.
Database	SQLite through better-sqlite3, WAL mode, plain SQL, no ORM	Zero setup, transactional, fast for this load. The schema is ordinary SQL, so moving to PostgreSQL is a port, not a rewrite. Trade-off: one writer.
API	Express 4, zod validation on every body, centralised error handler	Familiar and small. Validation errors return field-level messages the UI shows.

Decision	Choice	Why, and the trade-off
Frontend	React 18 and Vite, React Router, hand-written CSS with design tokens, no UI kit	A deliberate look and a small bundle (about 74 kB of gzipped JavaScript). Trade-off: more CSS to own.
Auth	bcrypt passwords, JWT in an httpOnly SameSite cookie, 12-hour session, role middleware	Tokens are not readable by page scripts. Trade-off: no server-side revocation yet.
Scoring	Deterministic rules engine instead of an LLM	Reproducible, explainable, testable, free to run, and CV data never leaves the system, which matters under the DPDP Act. Trade-off: lower recall on unusual wording (section 6.5).
File parsing	pdfjs-dist for PDF, mammoth for DOCX	I first used pdf-parse and found it can return the previous file's text in a long batch, so I replaced it and added a regression test.
Tests	Node's built-in test runner against the real app on an in-memory database	No test framework to install. Tests hit the real HTTP routes.

4. Data model

Six tables. `applications` is the join between a person and a role and holds everything about that pairing: the score, the evidence, the stage and the client's decision. One person can be ranked for several roles without being stored twice.

Table	Purpose	Key columns
clients	Companies that hire	id, name, sector
users	Sign-in accounts	email (unique), role (recruiter, client, head), password_hash, client_id (set for client users)
requisitions	A job to fill	client_id, title, jd_text, scorecard (JSON), created_at, first_submitted_at (drives the 48-hour metric)
candidates	One row per person	code (CND-91xxx), name, email, phone, headline, experience_years, skills (JSON), notice_days, location, expected_ctc, raw_text, file_hash, file_path, parse_status (parsed, scan, manual), consent_at, consent_source, consent_purpose, erased_at
applications	Candidate for a requisition	unique (requisition_id, candidate_id), score, confidence, breakdown (JSON: criteria with evidence and flags), stage, opened_at, submitted_at, ai_rank_at_submit, client_decision, reject_reason, interview_slot
audit_log	Who did what	at, actor_id, actor_name, actor_role, action, entity, entity_id, detail (JSON, IDs and codes only)

Why JSON columns? The scorecard and the score breakdown are always written and read as a whole and are never queried field by field. Everything that is filtered, joined or counted is a normal column.

5. Main flows

5.1 Job description to scorecard

1. The recruiter pastes the client's JD. The server extracts a title, experience band, notice limit, location, budget and skills.
2. Skills under "good to have" or "preferred" become bonus skills. Skills in the title get the highest weight.
3. Anything not found is listed as a note ("No notice period limit found"), never guessed.
4. The recruiter edits the scorecard and creates the requisition. They can edit it later, and every CV is re-scored.

5.2 Bulk upload and parsing

1. The browser sends files in batches of 20 so the recruiter sees real progress. The server accepts up to 100 per request and 5 MB per file, and the UI caps a session at 500.
2. Consent must be confirmed and its source chosen. Without it the request is refused.
3. For each file: the real format is detected from the file signature (not the extension), text is extracted, and a PDF with almost no text is treated as a scan.
4. The parser reads name, email, phone, role, years of experience (a stated figure, else employment date ranges with education years excluded), skills, notice period, location and CTC.
5. Duplicates are matched on email, phone or file hash. A duplicate is linked to the requisition instead of being stored twice.
6. In one transaction the candidate (with consent record), the application, the score and an audit entry are written. The result for each file is reported back: ranked, duplicate, already here, scan, or failed with a reason. One bad file never fails a batch.

5.3 Review gate and pipeline

Stages are sourced, screened, with client (submitted), shortlisted, interview, offered, joined and rejected. The server enforces who may move a candidate where. To move a candidate to **submitted** the recruiter must have opened the CV, and the candidate must have a score.

From	Recruiter can move to	Client can move to
sourced	screened, rejected	not visible
screened	sourced, submitted (CV opened first), rejected	not visible
submitted	screened (recall), rejected	shortlisted, interview, rejected
shortlisted	interview, rejected	interview, rejected
interview	offered, rejected	rejected
offered	joined, rejected	-
rejected	screened (reopen)	-

5.4 Client decision

The client sees only candidates submitted to their own company. They can shortlist, reject (a reason of at least 3 characters is required), or propose an interview slot (it must be in the future). The recruiter is told through a logged mock notification, and the decision, reason and slot appear on the recruiter's candidate panel.

5.5 Consent and erasure

Every candidate row has a consent timestamp, how consent was obtained, and the purpose ("Hiring and placement only"). Erasing a candidate (with a recorded reason) blanks the name, contact details, parsed profile and CV text, deletes the stored file, and stamps `erased_at`. The candidate disappears from every list, and their score and evidence are removed. The audit entry records only the candidate code and the reason.

5.6 Weekly sample for the delivery head

For each role with submissions in the last 7 days (up to 5), the page compares the recruiter's picks with the AI's top picks. If a recruiter submitted K candidates, agreement is how many of them were in the AI's top K. It then lists the disagreements in both directions: recruiter picked someone below the AI's top K, and the AI liked someone the recruiter did not submit. This is the data needed to tune the weights later (section 10).

6. How the AI part works

6.1 Pipeline

Text is extracted, parsed into fields, **anonymised**, then scored against the scorecard. Scoring uses only the anonymised text. The parser, anonymiser and scorer are pure functions with no database or network access, so they are fast to test and easy to reason about.

6.2 The score

A score is out of 100, normalised over the criteria the scorecard actually uses. Each criterion earns points and comes with evidence.

Group	Points	Rule
Must-have skills	50	Split by weight (1 to 3). Exact match (using aliases) earns full points. A related skill (for example Spring for Spring Boot) earns half and says so. Missing earns zero.
Bonus skills	10	Same rule, split equally.
Experience	20	Inside the band: full. Above the band: 5% less per year, floor 70%. Below: 25% less per year.
Notice period	10	Within the limit: full. Up to 30 days over: half. Beyond: 15%.
Location	5	Same city: full. Same Delhi NCR region: 70%. Elsewhere: 25%.
Budget (optional)	5	Expected CTC within budget: full. Up to 15% over: half. Beyond: 10%.

Unknown is not zero. If a fact cannot be found (no notice period, no stated experience, almost no text), the criterion is marked **unknown**, earns half credit, is flagged for a human, and lowers the confidence label. A thin CV is not punished as if it were a bad one.

Worked example (the sample candidates from the brief, scored against the example JD plus two bonus skills, Microservices and Hibernate, among 27 candidates):

Candidate	Score	Rank	Why
Ritika Bansal	94.7	5 of 27	Java, Spring Boot and Microservices found with quoted lines; 6.5 years inside 5 to 8; 60-day notice within the limit; Noida. Loses points only for no Hibernate.
Pooja Nair	59.2	18 of 27	Java and Hibernate found; no Spring Boot; 4 years is one below the band; 90-day notice is over the limit; Pune.
Arjun Rawat	24.7	27 of 27	No Java or Spring Boot anywhere in the CV (KYC profile); 3 years; 30-day notice is fine; Gurugram is in the same region.

6.3 Evidence

The evidence for a criterion is a whole line taken from the CV. Narrative bullets are preferred over bare skill lists. The matched term is highlighted, and in the CV tab every evidence line is highlighted in place. Because lines are selected by code rather than written by a model, **every quote exists verbatim in the CV**, and a test checks that for every matched criterion.

6.4 When the AI is wrong or unsure

Situation	What the system does
Anyone could be wrongly ranked low	Nobody is rejected automatically. The score only orders the list. Low matches sit in a collapsible group, stay searchable (search also covers the full CV text), and can still be reviewed and submitted. Low is shown in neutral grey, not red.
The reader over-trusts the number	Every criterion shows its evidence. A recruiter must open the CV before a candidate can be submitted, and the server enforces it. Each open is logged.
The parser misread a field	The Profile tab lets the recruiter correct role, years, notice, location and skills. All of that candidate’s applications are re-scored at once.
Information is missing	Marked unknown, half credit, flagged “Worth a human look”, confidence lowered.
The CV is a scanned image	Unscored and listed under “Needs a manual look”. The recruiter opens the original, pastes the text, and it is scored.
Bias through personal attributes	Name, gender, age, date of birth, marital status, photo, nationality and religion lines are removed before scoring, and the candidate’s name is replaced inside other lines. A test changes name, gender, age and marital status and asserts the score and every criterion status stay identical.

Situation	What the system does
Wording the taxonomy does not know	Related skills earn partial credit. The recruiter can add any skill to the scorecard, and it is matched literally. The weekly sample surfaces cases where recruiters disagreed with the AI.
The optional LLM is wrong or down	Its output is schema-validated and falls back to the rule-based extractor on any failure. Only the JD is sent, never a CV.

6.5 Known limits of the AI

- **It is matching, not understanding.** 53 skills are known (engineering, banking back-office, retail). Other skills must be typed into the scorecard and are matched literally. A candidate who describes Spring Boot work without the words “Spring Boot” will be missed.
- **It has not been measured on real CVs.** I tested on fictional CVs and seven generated sample files in PDF, DOCX and TXT. Multi-column layouts and tables in real PDFs may parse worse. The weekly sample is how accuracy would be measured in the pilot.
- **Weights are uncalibrated.** See A3.
- **Location and notice period can act as proxies for protected groups.** Both are optional criteria with small weights. A client who does not want them can leave them blank on the scorecard.
- **Experience is a heuristic.** A stated figure wins. Otherwise date ranges are summed, which can be wrong for unusual formats.

7. Security and privacy

Requirement	Control
Consent and purpose	Consent time, source and purpose are stored per candidate. Upload is refused without a consent confirmation (tested).
Deletion on request	Erasure removes personal data and the file, keeps an anonymous tombstone and an audit entry without personal data (tested).
Who viewed what	The audit log records profile views, CV opens, original-file opens, client views, decisions, stage changes and erasures. The delivery head can filter it. Entries hold IDs, candidate codes and any reason a user typed (the erase form asks them to leave out personal details), never contact details or CV text (tested).
Access control	Role checks in middleware. Every client query is scoped to the user’s own client, and another client’s record returns 404 so its existence is not revealed. The delivery head is read-only (all tested).
Data minimisation for clients	Masked name and contacts. CV text has emails, phones and profile links removed. Candidates never submitted to a client are invisible to them (tested).
Input validation	zod on every request body. Files are checked by signature, capped at 5 MB, and stored under a hash, never under the uploaded name. All SQL is parameterised.

Requirement	Control
Sessions	httpOnly SameSite=Lax cookie, optional Secure flag, 12-hour expiry, same-origin check on state-changing requests, login rate limit, and a dummy password comparison for unknown emails so timing does not reveal accounts.
Headers	helmet with a content security policy that allows only the app’s own scripts and Google Fonts.
Secrets	None in the repository. .env is git-ignored. In production the server refuses to start without JWT_SECRET. The demo password is for fictional demo accounts only.

Not done: encryption of personal data at rest beyond host disk encryption, MFA or SSO, session revocation, automatic retention purge, malware scanning of uploads, upload rate limits, and backups.

8. UI and design decisions

The brief asks for good, minimal design where nothing on screen lacks a purpose, and says recruiters scan many profiles and need the score, its evidence and the next action to be clear. Three ideas drive the design.

- **A score is a sum of reasons, so show the reasons.** Each candidate has a segmented meter. One segment per criterion, its width is the points available, and its fill is the points earned. Green is met, amber partly met, empty is not found, and striped is unknown. A recruiter reads strengths and gaps at a glance before reading a word, and the same meter appears in the list, the pipeline cards and the portal.
- **Separate what Sift says from what the CV says.** The interface uses a sans-serif (Manrope). Anything quoted from a CV uses a serif (Source Serif 4) in a tinted block with the matched term marked. You always know whose words you are reading.
- **One accent, used only for meaning.** Emerald means “matches”. Primary buttons are ink. Amber means partial or needs a human. Rose is reserved for destructive actions. A low match is neutral grey, because the AI does not reject anyone and the colour should not imply it does.

8.1 Key screens

Screen	What it shows and why
Sign in	A dark panel with a live miniature of one ranked row, which explains the product before a login. One-click demo accounts for reviewers (removable with VITE_SHOW_DEMO=false).
Requisitions	A stat strip, then one row per role with a reading-progress bar (“12 of 27 opened”). It makes the 60% screening-time goal visible: the fewer CVs opened per role, the better.
New requisition	Paste a JD, press Extract, edit the scorecard. Notes say what was not found. Must and Nice toggles with a three-step weight. The human always has the last word on what the AI extracted.

Screen	What it shows and why
Ranked list	Each row gives rank, name and role, three facts (years, notice, location), the meter, up to five skill chips with a status icon, the score, and a stage pill with the next step (“Open the CV”, “Ready to submit”). Lower matches are collapsed, not hidden. Scans get their own “Needs a manual look” group.
Candidate panel	A drawer so the list is never lost. Tabs: Score and evidence, CV (evidence highlighted, opening logged), Profile and privacy (corrections, consent, erase). The footer always offers one clear next action, and Submit is unavailable until the CV was opened.
Pipeline	A board from sourced to joined. Drag to move. Illegal moves are refused by the server with a plain message. Dropping on Rejected asks for a reason.
Client portal	Designed for a phone first. A single column, large touch targets, candidate cards with score and skills, a detail page with the evidence and the CV collapsed, and a sticky bottom bar for Reject, Propose slot and Shortlist. Reject and slot open as bottom sheets.
Insights and audit	Four KPIs with the brief’s targets beside them, the pipeline funnel, the weekly AI-vs-recruiter sample and the audit trail.

8.2 Visual system

Token	Value	Use
Paper	F6F7F9	Page background. Cool, slightly grey, so white cards read as surfaces.
Ink	172033	Text and primary buttons. A blue-leaning near-black, not pure black.
Ink 2	4B566B	Secondary text.
Emerald	0F7B5F	Match, met criteria, focus ring. The only accent.
Amber	B7791F	Partial match, low confidence, needs a human look.
Fog	A3ACBC	Not found, low match, borders on empty states.
Rose	B3382C	Destructive actions only.

- **Space and shape:** generous white space, 1 px borders instead of shadows (shadows appear only on drawers, modals and hover), corner radii of 16 for surfaces and 10 for controls.
- **Motion:** list rows rise in a short stagger, and meter segments fill once. Both respect the reduced-motion setting.
- **Every state is designed:** skeleton loaders, empty states that tell you the next step, error states with a retry, and a striped “unknown” state for information the AI could not verify.
- **Responsive:** the left rail becomes a slim top bar, the ranked row reflows into a card, the board scrolls with snap, modals become bottom sheets, and the portal bar respects the phone safe area.
- **Accessibility:** status is never colour alone (each has an icon and a text alternative), the meter has a text description of every criterion, visible focus outlines, Escape closes overlays, and form errors are announced.

9. Testing

Run `npm test`. The Node runner reports 40 tests: 39 test cases plus a shared setup hook.

Suite	Cases	What it proves
scoring.test.js	14	JD extraction of the example JD; parsing of dates, notice periods and experience; strong fit ranks above partial above off-profile; evidence is verbatim; related-skill partial credit; unknown is not zero; over and under experience; protected attributes are removed; score is identical when only name, gender, age or marital status change.
ingest.test.js	4	File type detection by signature; a mixed batch of PDF and DOCX files parses without mixing contents (the bug that made me drop pdf-parse); scans yield no text; corrupt PDFs fail.
api.test.js	21	Login and roles; clients cannot reach staff routes and cannot see each other’s candidates; unsubmitted candidates are invisible to clients; contacts are masked; ranking order and low scorers listed; review gate; illegal stage jumps; reject reason and future slot rules; consent required; duplicates; scans; bad files; scorecard re-scoring and validation; erasure; audit trail; weekly sample.

Not covered: the front end has no automated tests in the repository. During development its main flows were smoke-tested in a simulated browser DOM against the real server, and they should still be checked on real phones and browsers. There are no end-to-end tests and no load tests.

10. What I cut, and what I would do next

10.1 Cut

Cut	Why and what happens instead
OCR for scanned CVs	Needs a rendering and OCR pipeline. Scans are flagged, unscored and listed for manual handling, with a paste-the-text path that then scores them.
Real email and WhatsApp	The brief allows mocking. Decisions create logged mock notifications.
Candidate-facing consent link	Consent is recorded by the recruiter at upload (A6).
Automatic retention purge	Erasure on request is built. A schedule is not.
SSO, MFA, session revocation	Out of the six-week slice.
Background job queue	Parsing runs inside the request, in batches of 20. Fine for 500 CVs, not for thousands at once.
Shortlist export to Excel or PDF	The portal replaces the emailed Excel sheet. Export is an easy add.
Scorecard templates and versions	Each requisition has its own scorecard (A11).

10.2 Next, in order of impact on the three success metrics

1. **Calibrate the weights** using the weekly sample as labelled data (recruiter picks and client decisions). This raises client acceptance toward 60%.
2. **OCR for scans,** so no CV needs manual transcription.
3. **An LLM as a second reader.** It would read the anonymised text and propose evidence for criteria the rules missed. Every quote would be checked as a substring of the CV before being shown, so the verifiable-evidence guarantee stays.
4. **PostgreSQL and a job queue,** for more than one instance and bigger batches.
5. **Candidate consent link and retention rules,** reviewed by counsel.
6. **Real notifications, SSO and MFA.**
7. **Browser end-to-end tests** and a real-device pass on the client portal.

11. Success metrics: how the product measures them

Target from the brief	How the Insights page measures it
Screening time per requisition down 60%	Reading time saved = (CVs ranked minus CVs a recruiter opened) × 3 minutes (A1). A recruiter opens the top of the list instead of every CV.
Shortlist within 48 hours for 80% of requisitions	Share of requisitions where first_submitted_at minus created_at is 48 hours or less, plus the average hours.
Client acceptance from 45% to 60%	Candidates the client shortlisted or took to interview divided by all candidates the client decided on, shown beside the 45% baseline and 60% target.

The figures on the demo are computed from seeded, fictional data so the page is not empty. They are not evidence of real performance.

12. Repository

- `server/`: Express API, SQLite, scoring engine and tests. `client/`: React app. `sample-cvs/`: fictional CVs for trying bulk upload, one for each outcome (ranked, duplicate, scan, corrupt).
- History uses small commits with conventional prefixes (feat, test, docs, chore), in the order the system was built: parsing, scoring, ingestion, APIs, seed data, tests, then the interface.
- Formatting is enforced with Prettier (`npm run format:check`). Modules are small and single-purpose, and the engine is separated from HTTP and from the database.